

**TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
ĐẠI HỌC QUỐC GIA TP.HCM**



**BÁO CÁO ĐỒ ÁN CUỐI KÌ
HỆ THỐNG GỢI Ý NỘP BÀI BÁO
KHOA HỌC**

Giảng viên hướng dẫn: PGS.TS. Nguyễn Thanh Bình
Thành viên:

Huỳnh Minh Thuận	- 22110217
Trương Minh Thuật	- 22110218
Nguyễn Minh Trí	- 22110235
Nguyễn Phạm Anh Trí	- 22110236

TP.HCM, tháng 1 năm 2025

Lời cảm ơn

Trước hết, tôi xin bày tỏ lòng biết ơn sâu sắc đến giảng viên hướng dẫn đã tận tình chỉ dẫn, góp ý và định hướng xuyên suốt quá trình thực hiện đề tài. Những nhận xét mang tính học thuật và thực tiễn của thầy/cô không chỉ giúp tôi hoàn thiện nội dung báo cáo, mà còn giúp tôi tiếp cận vấn đề một cách hệ thống và đúng đắn hơn trong lĩnh vực kỹ nghệ dữ liệu và trí tuệ nhân tạo.

Tôi cũng xin chân thành cảm ơn Khoa/Bộ môn đã tạo điều kiện thuận lợi về môi trường học tập, tài nguyên và cơ sở hạ tầng, giúp tôi có cơ hội triển khai đề tài với quy mô và chiều sâu kỹ thuật phù hợp. Các học phần nền tảng và chuyên ngành đã cung cấp cho tôi những kiến thức cần thiết để hình thành tư duy thiết kế và triển khai một hệ thống dữ liệu hoàn chỉnh.

Bên cạnh đó, tôi xin gửi lời cảm ơn đến các nguồn dữ liệu học thuật mở như OpenAlex, Scopus và Springer, cũng như cộng đồng mã nguồn mở, những nền tảng đã cung cấp dữ liệu, công cụ và thư viện quan trọng, góp phần không nhỏ vào việc hiện thực hóa hệ thống gợi ý trong đề tài này.

Cuối cùng, tôi xin cảm ơn gia đình, bạn bè và các bạn học cùng lớp đã luôn động viên, hỗ trợ và tạo điều kiện về mặt tinh thần trong suốt quá trình học tập và thực hiện dự án. Sự đồng hành đó là nguồn động lực quan trọng giúp tôi hoàn thành đề tài này đúng tiến độ và với tinh thần nghiêm túc nhất.

Mặc dù đã có nhiều cố gắng, báo cáo khó tránh khỏi những hạn chế và thiếu sót nhất định. Tôi rất mong nhận được các ý kiến đóng góp quý báu từ thầy cô và người đọc để có thể tiếp tục hoàn thiện và phát triển đề tài trong tương lai.

Mục lục

Lời cảm ơn	1
1 Giới thiệu và động lực nghiên cứu	6
1.1 Bối cảnh nghiên cứu	6
1.2 Những thách thức hiện hữu	6
1.3 Động lực xây dựng hệ thống	6
2 Phát biểu bài toán và mục tiêu	8
2.1 Mô tả bài toán thực tế	8
2.1.1 Đầu vào (Input)	9
2.1.2 Xử lý (Process)	9
2.1.3 Đầu ra (Output)	9
2.2 Mục tiêu kỹ thuật của dự án	10
2.2.1 Xây dựng đường ống dữ liệu tự động (Automated ETL Pipeline)	10
2.2.2 Phát triển mô hình gợi ý dựa trên ngữ nghĩa (Semantic Recommendation)	10
2.2.3 Hoàn thiện kiến trúc Full-stack cho hệ thống	11
3 Kiến trúc tổng thể hệ thống	12
3.1 Sơ đồ kiến trúc tổng thể (High-level System Architecture)	12
3.2 Tầng thu thập dữ liệu (Data Ingestion Layer)	13
3.3 Tầng lưu trữ dữ liệu thô (Data Lake Layer)	13
3.4 Tầng xử lý và chuẩn hóa dữ liệu (Data Processing & Normalization Layer)	14
3.5 Tầng vector hóa và tìm kiếm ngữ nghĩa (Semantic Embedding Layer) . .	14
3.6 Tầng điều phối quy trình xử lý (Workflow Orchestration Layer)	14
3.7 Tầng ứng dụng và tương tác người dùng (Application Layer)	15
3.8 Vận hành và giám sát hệ thống (System Operations & Monitoring) . . .	15
4 Thu thập và xử lý dữ liệu	16
4.1 Tổng quan quy trình ETL của hệ thống	16
4.2 Thu thập dữ liệu thô (Data Ingestion)	16
4.2.1 Nguồn dữ liệu học thuật	16
4.2.2 Cơ chế thu thập dữ liệu tự động	17
4.3 Lưu trữ dữ liệu thô tại vùng đệm (Staging Layer)	17
4.4 Biến đổi và chuẩn hóa dữ liệu (Data Transformation & Normalization) .	17
4.4.1 Xử lý dữ liệu bằng Pandas	17

4.4.2	Ảnh xạ vào mô hình dữ liệu quan hệ	18
5	Kho dữ liệu và quản trị dữ liệu (Data Warehouse)	19
5.1	Tổng quan kiến trúc lưu trữ trung tâm	19
5.2	Thiết kế lược đồ dữ liệu quan hệ	20
5.3	Lưu trữ và quản lý dữ liệu vector ngữ nghĩa	21
5.4	Chiến lược tối ưu hóa hiệu năng kho dữ liệu	21
6	Điều phối và tự động hóa xử lý (Pipelines & Orchestration)	22
6.1	Mô hình điều phối dựa trên đồ thị DAG	22
6.2	Các giai đoạn xử lý	22
6.3	Quản lý phụ thuộc và đảm bảo tính nhất quán dữ liệu	23
7	Mô hình đề xuất hội nghị và tạp chí khoa học (Conference & Journal Recommendation Model)	24
7.1	Data Preprocessing	24
7.1.1	Preprocessing cho trường Title và Abstract	25
7.1.2	Preprocessing cho trường Keywords	25
7.2	Content-Based Filtering using Weighted Vector Space Model	25
7.2.1	Feature Selection with Chi-Square Statistic	25
7.2.2	TF-IDF Vectorization & Multi-field Weighting	26
7.2.3	Online Inference & Recommendation	26
7.3	Hybrid Search Strategy & Reciprocal Rank Fusion (RRF)	26
7.3.1	Data Embedding Generation (Offline Pipeline)	27
7.3.2	User Input Processing (Online Inference)	27
7.3.3	Retrieval & Ranking Mechanism	28
8	Phát triển ứng dụng web (Web Application)	29
8.1	Kiến trúc tổng thể ứng dụng	29
8.2	Tầng Backend và tổ chức xử lý nghiệp vụ	29
8.3	Tầng Frontend và giao diện người dùng	29
8.4	Quản lý độ trễ và khả năng diễn giải kết quả	30
9	Triển khai và vận hành hệ thống (DevOps)	31
9.1	Chiến lược đóng gói và đảm bảo tính nhất quán môi trường	31
9.2	Điều phối dịch vụ với Docker Compose	31
9.3	Tự động hóa quy trình triển khai	32
9.4	Quản lý dữ liệu và bảo mật cấu hình	32
9.5	Đánh giá khả năng mở rộng ở mức thiết kế	32
10	Hạn chế và hướng phát triển	33
10.1	Hạn chế của hệ thống	33
10.2	Hướng phát triển trong tương lai	33
11	Kết luận và đánh giá	35
11.1	Những kết quả và đóng góp đạt được	35

11.2	Đánh giá hệ thống	35
12	Lời kết	37
	Tài liệu tham khảo	38

Danh sách hình vẽ

2.1	Mô tả	8
3.1	Mô tả	12
5.1	Lược đồ thực thể-quan hệ của kho dữ liệu	20
6.1	Lược đồ thực thể-quan hệ của kho dữ liệu	22

Chương 1

Giới thiệu và động lực nghiên cứu

1.1 Bối cảnh nghiên cứu

Trong kỷ nguyên số hóa hiện nay, số lượng các công bố khoa học đang tăng trưởng theo cấp số nhân. Sự bùng nổ của các cơ sở dữ liệu học thuật như Scopus, Springer, hay OpenAlex đã tạo ra một kho tàng tri thức khổng lồ nhưng cũng đồng thời gây ra hiện tượng "quá tải thông tin" đối với cộng đồng nghiên cứu. Việc lựa chọn một tạp chí (Journal) hoặc hội nghị (Conference) phù hợp để nộp bài báo (Paper submission) đã trở thành một quy trình phức tạp, đòi hỏi sự hiểu biết sâu sắc về phạm vi nghiên cứu (Aims & Scope), chỉ số ảnh hưởng (Impact Factor) và tính tương thích về nội dung của bản thảo.

1.2 Những thách thức hiện hữu

Thực tế cho thấy, việc gửi bài báo đến các đơn vị xuất bản không phù hợp thường dẫn đến kết quả bị từ chối ngay từ vòng sơ loại (Desk rejection). Điều này không chỉ gây lãng phí thời gian quý báu của các tác giả (thường kéo dài từ vài tuần đến vài tháng) mà còn làm chậm tiến độ phổ biến các kết quả nghiên cứu mới ra cộng đồng. Các phương pháp tìm kiếm thủ công truyền thống dựa trên từ khóa đơn giản thường không phản ánh được sự phức tạp về ngữ nghĩa và tính liên ngành của các nghiên cứu hiện đại.

1.3 Động lực xây dựng hệ thống

Xuất phát từ những thách thức trên, nhu cầu về một hệ thống hỗ trợ quyết định thông minh, có khả năng tự động phân tích và gợi ý địa điểm xuất bản phù hợp là vô cùng cấp thiết. Động lực chính của nghiên cứu này tập trung vào ba yếu tố cốt lõi:

- **Tối ưu hóa quy trình học thuật:** Giảm thiểu thời gian và công sức cho nhà nghiên cứu trong việc sàng lọc hàng ngàn tạp chí tiềm năng.
- **Ứng dụng kỹ thuật xử lý ngôn ngữ tự nhiên (NLP) tiên tiến:** Tận dụng các mô hình ngôn ngữ lớn và kỹ thuật Vector Embedding để trích xuất đặc trưng ngữ nghĩa từ tiêu đề và tóm tắt bài báo, thay vì chỉ so khớp từ khóa đơn thuần.

- **Xây dựng luồng dữ liệu tự động (Automated Data Pipeline):** Thiết lập một hệ thống toàn diện từ khâu thu thập dữ liệu (Scraping), chuẩn hóa (Normalization) đến lưu trữ và xử lý, đảm bảo tính cập nhật và độ tin cậy của thông tin gợi ý.

Hệ thống **Project Paper Submission Recommendation** không chỉ dừng lại ở một ứng dụng Web đơn lẻ, mà còn là một minh chứng cho việc kết hợp giữa kỹ nghệ dữ liệu (Data Engineering) và trí tuệ nhân tạo để giải quyết một bài toán thực tiễn trong quản trị tri thức khoa học.

Chương 2

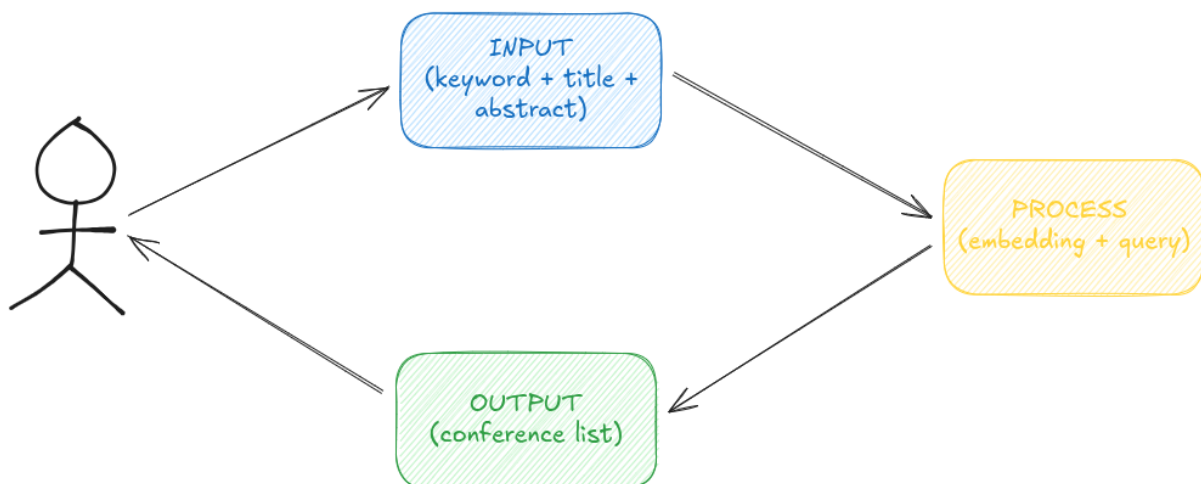
Phát biểu bài toán và mục tiêu

2.1 Mô tả bài toán thực tế

Bài toán được đặt ra trong bối cảnh thực tế của các tác giả bài báo khoa học khi chuẩn bị công bố kết quả nghiên cứu. Ở giai đoạn hoàn thiện bản thảo, một trong những quyết định quan trọng nhất là lựa chọn tạp chí hoặc hội nghị khoa học phù hợp nhất để nộp bài. Quyết định này ảnh hưởng trực tiếp đến khả năng được chấp nhận, thời gian phản biện, cũng như mức độ lan tỏa học thuật của công trình.

Tuy nhiên, trong thực tế, các tác giả – đặc biệt là sinh viên và nhà nghiên cứu trẻ – thường gặp khó khăn do thiếu kinh nghiệm, thiếu thông tin tổng quan về các tạp chí trong cùng lĩnh vực, hoặc không có công cụ hỗ trợ phân tích mức độ phù hợp giữa nội dung bài báo và phạm vi (scope) của từng venue. Việc tìm kiếm thủ công dựa trên từ khóa hoặc tham khảo danh sách tạp chí liên quan thường mang tính cảm tính, tốn thời gian và không đảm bảo độ chính xác.

Xuất phát từ kịch bản sử dụng đó, hệ thống được thiết kế để hỗ trợ **use-case cốt lõi** sau:



Hình 2.1: Mô tả

2.1.1 Đầu vào (Input)

Người dùng cung cấp các thông tin mô tả nội dung học thuật của bản thảo, bao gồm:

- **Tiêu đề (Title):** Tóm lược ngắn gọn chủ đề nghiên cứu.
- **Từ khóa (Keyword):** Một danh sách các từ khóa liên quan đến chủ đề của bài nghiên cứu.
- **Tóm tắt (Abstract):** Mô tả chi tiết mục tiêu, phương pháp và kết quả chính của bài báo.

Các thành phần này được xem là đại diện ngữ nghĩa chính cho nội dung nghiên cứu, đồng thời là dữ liệu phổ biến và sẵn có ngay cả khi bài báo chưa hoàn thiện.

2.1.2 Xử lý (Process)

Từ dữ liệu đầu vào, hệ thống thực hiện chuỗi xử lý nhằm chuyển đổi nội dung văn bản thành dạng có thể so sánh và xếp hạng:

- Phân tích và tiền xử lý văn bản (loại bỏ nhiễu, chuẩn hóa).
- Biểu diễn nội dung bài báo dưới dạng **vector đặc trưng ngữ nghĩa (feature/embedding vector)**.
- So sánh vector này với tập vector đại diện cho các tạp chí/hội nghị đã được thu thập, lập chỉ mục và lưu trữ trước đó trong hệ thống.
- Tính toán độ tương đồng nội dung giữa bài báo và từng venue dựa trên các thước đo phù hợp.

Quá trình này cho phép hệ thống đánh giá mức độ phù hợp một cách định lượng, thay vì dựa trên suy đoán chủ quan.

2.1.3 Đầu ra (Output)

Kết quả trả về cho người dùng:

- **Danh sách xếp hạng (Ranked List)** các tạp chí/hội nghị được đề xuất, sắp xếp theo mức độ phù hợp giảm dần.
- Thông tin hỗ trợ ra quyết định, bao gồm:
 - Mức độ tương đồng nội dung.
 - Tần suất các bài báo có chủ đề tương tự đã được xuất bản tại venue đó.
 - Một số metadata cơ bản (lĩnh vực, nhà xuất bản, chu kỳ xuất bản).

Danh sách này đóng vai trò như một công cụ hỗ trợ quyết định, giúp người dùng nhanh chóng khoanh vùng các venue tiềm năng thay vì phải tìm kiếm thủ công.

2.2 Mục tiêu kỹ thuật của dự án

Để giải quyết bài toán trên một cách có hệ thống và khả thi trong thực tế, dự án tập trung vào ba mục tiêu kỹ thuật cốt lõi sau:

2.2.1 Xây dựng đường ống dữ liệu tự động (Automated ETL Pipeline)

Dữ liệu học thuật là nền tảng của hệ thống gợi ý. Do đó, dự án hướng tới:

- Thiết kế cơ chế **thu thập dữ liệu tự động (Scraping / API ingestion)** từ các nguồn học thuật mở như OpenAlex, Scopus.
- Thu thập các thông tin liên quan đến tạp chí và bài báo đã xuất bản (tiêu đề, abstract, lĩnh vực, venue).
- Áp dụng các bước **chuẩn hóa (normalization)** và **làm sạch dữ liệu (data cleaning)** nhằm đảm bảo tính nhất quán trước khi lưu trữ vào kho dữ liệu trung tâm.
- Hình thành pipeline có khả năng mở rộng và tái sử dụng, phục vụ cập nhật dữ liệu định kỳ.

Mục tiêu là giảm tối đa can thiệp thủ công và đảm bảo dữ liệu luôn sẵn sàng cho các bước phân tích tiếp theo.

2.2.2 Phát triển mô hình gợi ý dựa trên ngữ nghĩa (Semantic Recommendation)

Thay vì dựa trên so khớp từ khóa đơn thuần, dự án tập trung vào hiểu **ngữ nghĩa nội dung**:

- Tích hợp dịch vụ **embedding** để chuyển đổi văn bản (title, abstract) thành vector phản ánh ngữ cảnh và ý nghĩa.
- Lưu trữ các vector đại diện cho bài báo và tạp chí trong hệ thống tìm kiếm phù hợp.
- Triển khai các thuật toán **tính toán độ tương đồng (similarity search)** hiệu quả, có khả năng hoạt động trên tập dữ liệu lớn.
- Đảm bảo thời gian truy vấn đủ nhanh để phục vụ người dùng cuối trong môi trường web.

Mục tiêu của phần này là tạo ra các gợi ý có tính học thuật cao, phản ánh đúng mức độ tương thích về nội dung nghiên cứu.

2.2.3 Hoàn thiện kiến trúc Full-stack cho hệ thống

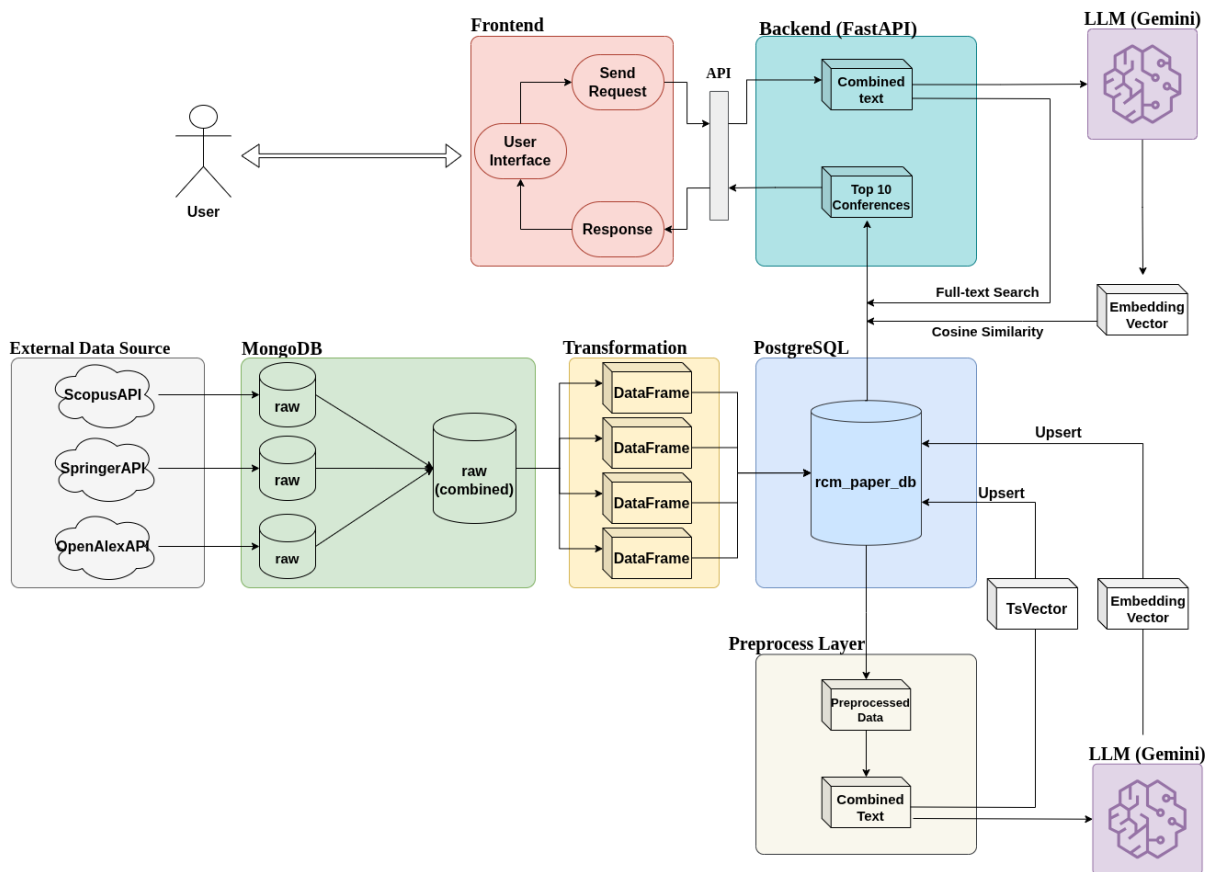
Để hệ thống có thể sử dụng trong thực tế, dự án hướng tới một kiến trúc hoàn chỉnh:

- Xây dựng **Backend API** có khả năng tiếp nhận yêu cầu, xử lý embedding, truy vấn dữ liệu và trả về kết quả gợi ý.
- Đảm bảo hiệu năng, khả năng mở rộng và tính ổn định của dịch vụ.
- Phát triển **Frontend Web Application** với giao diện trực quan, cho phép người dùng nhập thông tin bài báo và xem kết quả đề xuất một cách dễ hiểu.
- Tạo cầu nối liên mạch giữa người dùng và các thành phần xử lý phức tạp phía sau.

Chương 3

Kiến trúc tổng thể hệ thống

3.1 Sơ đồ kiến trúc tổng thể (High-level System Architecture)



Hình 3.1: Mô tả

Hình trên minh họa kiến trúc tổng thể của hệ thống gợi ý nộm bài báo khoa học, được thiết kế theo hướng **Data-Centric Architecture**. Kiến trúc này mô tả đầy đủ luồng dữ liệu từ giai đoạn thu thập dữ liệu học thuật thô, xử lý – chuẩn hóa, cho đến khi phục vụ truy vấn gợi ý từ phía người dùng cuối.

Toàn bộ hệ thống được chia thành các lớp chức năng rõ ràng, bao gồm:

- Tầng thu thập dữ liệu (Ingestion Layer)
- Tầng lưu trữ dữ liệu (Storage Layer)
- Tầng xử lý và chuẩn hóa (Processing Layer)
- Tầng vector hóa và tìm kiếm ngữ nghĩa (Semantic Layer)
- Tầng điều phối và vận hành (Orchestration & Operations)
- Tầng ứng dụng và tương tác người dùng (Application Layer)

Các tầng này liên kết với nhau thông qua một pipeline xử lý dữ liệu khép kín, được điều phối tự động và triển khai trong môi trường container hóa.

3.2 Tầng thu thập dữ liệu (Data Ingestion Layer)

Tầng Ingestion chịu trách nhiệm thu thập dữ liệu học thuật từ các nguồn mở như OpenAlex, Scopus và Springer.

Các module thu thập dữ liệu được thiết kế độc lập theo từng nguồn, đảm bảo:

- Khả năng mở rộng khi bổ sung nguồn mới.
- Không ảnh hưởng đến pipeline xử lý phía sau.

Dữ liệu thu thập bao gồm thông tin bài báo, tạp chí/hội nghị, tác giả và các metadata liên quan. Tất cả dữ liệu ở giai đoạn này được xem là dữ liệu thô và chưa áp dụng bất kỳ giả định nào về cấu trúc.

3.3 Tầng lưu trữ dữ liệu thô (Data Lake Layer)

Dữ liệu sau khi thu thập được lưu trữ trực tiếp vào **MongoDB**, đóng vai trò Data Lake của hệ thống.

Việc sử dụng MongoDB mang lại các lợi ích:

- Lưu trữ linh hoạt dữ liệu bán cấu trúc.
- Không áp đặt schema cố định.
- Bảo toàn toàn bộ dữ liệu gốc để phục vụ tái xử lý.

Tầng này hoạt động như **Staging Area**, tách biệt hoàn toàn với dữ liệu phục vụ truy vấn và phân tích.

3.4 Tầng xử lý và chuẩn hóa dữ liệu (Data Processing & Normalization Layer)

Từ Data Lake, dữ liệu được trích xuất và xử lý bằng các script Python sử dụng thư viện **Pandas**.

Các bước xử lý bao gồm:

- Làm sạch dữ liệu (loại bỏ bản ghi lỗi, trùng lặp).
- Xử lý dữ liệu thiếu và không nhất quán.
- Chuẩn hóa dữ liệu theo schema định nghĩa trước.
- Ánh xạ dữ liệu thành các thực thể logic như *Paper*, *Author*, *Venue*.

Dữ liệu sau xử lý được nạp vào **PostgreSQL**, nơi đóng vai trò kho dữ liệu có cấu trúc của hệ thống.

3.5 Tầng vector hóa và tìm kiếm ngữ nghĩa (Semantic Embedding Layer)

Ở tầng này, hệ thống tiến hành làm giàu dữ liệu bằng các kỹ thuật AI.

Các trường văn bản quan trọng (Title, Abstract) được đưa vào mô hình **Embedding** để tạo vector biểu diễn ngữ nghĩa. Các vector này được lưu trữ trực tiếp trong PostgreSQL thông qua **pgvector**.

Tầng này cho phép:

- Tìm kiếm tương đồng ngữ nghĩa giữa bài báo và venue.
- Xếp hạng kết quả dựa trên độ tương đồng nội dung.
- Triển khai hệ thống gợi ý mà không cần Vector Database riêng biệt

3.6 Tầng điều phối quy trình xử lý (Workflow Orchestration Layer)

Apache Airflow đảm nhiệm vai trò điều phối các pipeline xử lý dữ liệu.

Tầng điều phối chịu trách nhiệm:

- lập lịch thu thập dữ liệu,
- kích hoạt các tác vụ xử lý – chuẩn hóa,
- điều phối quá trình vector hóa,
- giám sát và xử lý lỗi trong pipeline.

Tầng này đảm bảo tính nhất quán và độ tin cậy của toàn bộ luồng dữ liệu.

3.7 Tầng ứng dụng và tương tác người dùng (Application Layer)

Tầng ứng dụng cung cấp giao diện tương tác cho người dùng cuối.

- **Backend (FastAPI)** tiếp nhận yêu cầu, thực hiện vector hóa nội dung đầu vào và truy vấn PostgreSQL để sinh danh sách gợi ý.
- **Frontend (Next.js)** hiển thị kết quả và hỗ trợ tương tác trực quan.

3.8 Vận hành và giám sát hệ thống (System Operations & Monitoring)

Khác với các tầng chức năng phía trên, **vận hành hệ thống** là một mối quan tâm xuyên suốt, bao phủ toàn bộ kiến trúc.

Nội dung vận hành bao gồm:

- Triển khai hệ thống bằng Docker và Docker Compose.
- Quản lý vòng đời container và tài nguyên hệ thống.
- Giám sát trạng thái các thành phần (database, pipeline, API).
- Hỗ trợ mở rộng và bảo trì hệ thống trong môi trường thực tế.

Việc đặt tầng vận hành ở cuối chương giúp nhấn mạnh vai trò của nó như một lớp bao trùm, đảm bảo hệ thống hoạt động ổn định và bền vững.

Chương 4

Thu thập và xử lý dữ liệu

4.1 Tổng quan quy trình ETL của hệ thống

Trong hệ thống gợi ý nộp bài báo khoa học, dữ liệu học thuật đóng vai trò nền tảng, quyết định trực tiếp đến chất lượng của các kết quả đề xuất. Do đó, dự án tập trung xây dựng một **đường ống ETL (Extract – Transform – Load) tự động**, có khả năng thu thập dữ liệu từ nhiều nguồn học thuật khác nhau, xử lý và chuẩn hóa dữ liệu một cách nhất quán trước khi đưa vào các tầng xử lý phía sau.

Về mặt tổng thể, quy trình ETL của hệ thống bao gồm ba giai đoạn chính:

- **Thu thập dữ liệu thô (Data Ingestion)** từ các nguồn học thuật mở.
- **Biến đổi và chuẩn hóa dữ liệu (Data Transformation & Normalization)** nhằm đảm bảo tính nhất quán và toàn vẹn.
- **Nạp dữ liệu có cấu trúc (Data Loading)** vào hệ quản trị cơ sở dữ liệu quan hệ để phục vụ truy vấn và phân tích.

Quy trình này được tự động hóa và điều phối tập trung nhằm đảm bảo khả năng mở rộng, tái sử dụng và dễ dàng giám sát.

4.2 Thu thập dữ liệu thô (Data Ingestion)

4.2.1 Nguồn dữ liệu học thuật

Hệ thống thu thập dữ liệu từ các nền tảng học thuật uy tín và phổ biến như **OpenAlex**, **Scopus** và **Springer**. Đây là các nguồn cung cấp thông tin có cấu trúc về bài báo khoa học, tạp chí và hội nghị, bao gồm tiêu đề, tóm tắt, danh sách tác giả, lĩnh vực nghiên cứu và thông tin xuất bản.

Việc sử dụng nhiều nguồn dữ liệu khác nhau giúp hệ thống giảm thiểu thiên lệch và mở rộng phạm vi bao phủ của các lĩnh vực nghiên cứu.

4.2.2 Cơ chế thu thập dữ liệu tự động

Hệ thống triển khai các bộ thu thập dữ liệu (Scrapers) tương tác trực tiếp với API chính thức của từng nguồn. Các Scrapers được thiết kế theo hướng mô-đun, cho phép dễ dàng mở rộng hoặc thay thế khi có nguồn dữ liệu mới.

Một số đặc điểm chính của cơ chế thu thập bao gồm:

- **Tần suất cập nhật:** Dữ liệu được thu thập theo chu kỳ hàng ngày nhằm đảm bảo hệ thống luôn phản ánh kịp thời các thay đổi trong hoạt động xuất bản khoa học.
- **Quản lý giới hạn truy cập:** Các Scrapers được cấu hình cơ chế kiểm soát tốc độ truy cập (rate limiting) và xử lý lỗi kết nối để tuân thủ chính sách của các nhà cung cấp dữ liệu.
- **Khả năng chịu lỗi:** Trong trường hợp một nguồn dữ liệu tạm thời không khả dụng, hệ thống ghi nhận trạng thái lỗi và tiếp tục xử lý các nguồn còn lại.

4.3 Lưu trữ dữ liệu thô tại vùng đệm (Staging Layer)

Toàn bộ dữ liệu thu thập được từ các nguồn học thuật được lưu trữ trực tiếp vào **MongoDB** dưới dạng tài liệu (document).

Việc lựa chọn MongoDB xuất phát từ các lý do sau:

- **Tính linh hoạt về schema:** Dữ liệu trả về từ các nguồn khác nhau có cấu trúc không đồng nhất. MongoDB cho phép lưu trữ dữ liệu ở dạng bán cấu trúc (semi-structured) mà không cần chuẩn hóa ngay từ đầu.
- **Bảo toàn dữ liệu gốc:** Dữ liệu được lưu giữ nguyên bản nhằm phục vụ cho mục đích kiểm tra, đối chiếu hoặc tái xử lý trong tương lai.
- **Vai trò vùng đệm (staging):** MongoDB đóng vai trò như một lớp trung gian giữa quá trình thu thập và xử lý, giúp tách biệt rõ ràng hai giai đoạn Extract và Transform trong pipeline.

4.4 Biến đổi và chuẩn hóa dữ liệu (Data Transformation & Normalization)

4.4.1 Xử lý dữ liệu bằng Pandas

Sau khi dữ liệu thô được lưu trữ tại vùng đệm, hệ thống tiến hành xử lý và chuẩn hóa dữ liệu bằng các script Python sử dụng thư viện **Pandas**. Đây là giai đoạn quan trọng nhằm chuyển dữ liệu từ dạng phi cấu trúc sang dạng có cấu trúc chặt chẽ, sẵn sàng cho việc lưu trữ và truy vấn.

Các bước xử lý chính bao gồm:

- **Làm sạch dữ liệu:** Loại bỏ các bản ghi không đầy đủ hoặc không hợp lệ; xử lý các giá trị thiếu trong các trường quan trọng như tiêu đề và tóm tắt bài báo.
- **Khử trùng lặp:** Sử dụng các định danh duy nhất (ví dụ: DOI) để đảm bảo mỗi bài báo chỉ tồn tại một bản ghi duy nhất trong hệ thống.
- **Chuẩn hóa dữ liệu:** Đưa các trường thông tin như ngày xuất bản, tên tác giả và tên tạp chí về định dạng thống nhất nhằm giảm sự không nhất quán giữa các nguồn.

4.4.2 Ánh xạ vào mô hình dữ liệu quan hệ

Dữ liệu sau khi được xử lý và chuẩn hóa sẽ được ánh xạ vào **schema quan hệ** đã được thiết kế trong hệ quản trị cơ sở dữ liệu **PostgreSQL**. Quá trình này đảm bảo:

- Tính toàn vẹn tham chiếu giữa các thực thể như *Paper*, *Author* và *Venue*.
- Khả năng truy vấn hiệu quả phục vụ cho các tầng xử lý và gợi ý phía sau.
- Nền tảng dữ liệu ổn định cho việc mở rộng hệ thống trong tương lai.

Chương 5

Kho dữ liệu và quản trị dữ liệu (Data Warehouse)

5.1 Tổng quan kiến trúc lưu trữ trung tâm

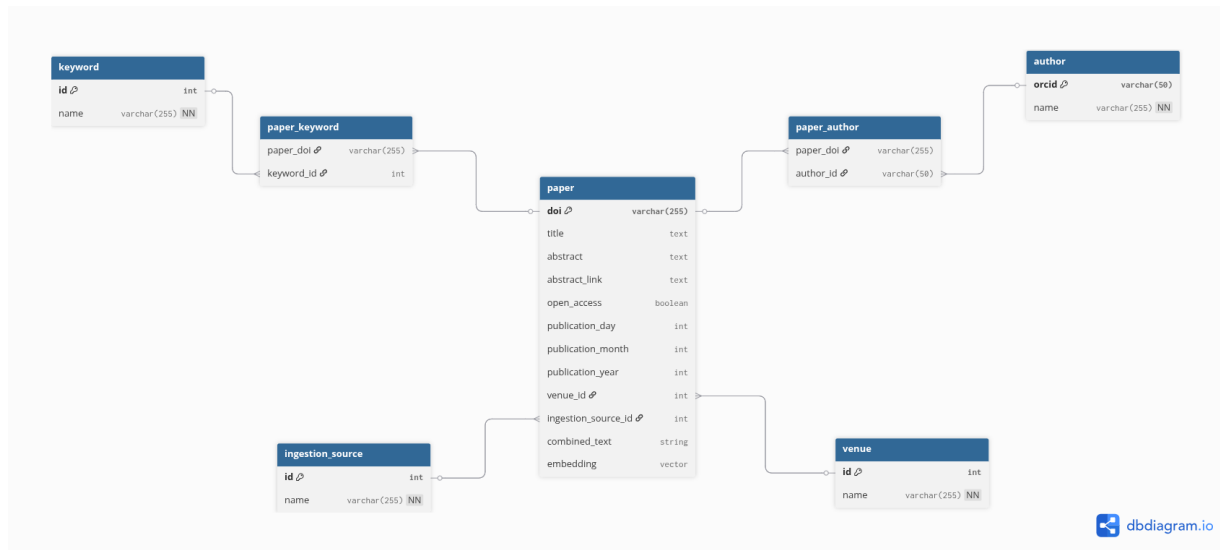
Trong kiến trúc định hướng dữ liệu của hệ thống, (**Kho dữ liệu (Data Warehouse)**) giữ vai trò hạt nhân, chịu trách nhiệm lưu trữ dữ liệu đã qua làm sạch và chuẩn hóa, phục vụ trực tiếp cho các tác vụ truy vấn, phân tích và hệ thống gợi ý.

Khác với tầng dữ liệu thô (*staging area*) sử dụng MongoDB để lưu trữ dữ liệu bán cấu trúc, kho dữ liệu được xây dựng trên nền tảng **PostgreSQL**, một hệ quản trị cơ sở dữ liệu quan hệ có độ tin cậy cao và khả năng mở rộng linh hoạt. Việc lựa chọn PostgreSQL cho tầng lưu trữ trung tâm xuất phát từ các lý do kỹ thuật chính sau:

- **Đảm bảo toàn vẹn và nhất quán dữ liệu:** PostgreSQL tuân thủ đầy đủ các tính chất ACID, cho phép quản lý an toàn các mối quan hệ phức tạp giữa bài báo, tác giả và nơi xuất bản.
- **Hỗ trợ đa dạng mô hình dữ liệu:** Bên cạnh mô hình quan hệ, PostgreSQL hỗ trợ dữ liệu JSON và dữ liệu vector thông qua phần mở rộng *pgvector*, cho phép tích hợp dữ liệu nghiệp vụ và dữ liệu ngữ nghĩa trong cùng một hệ thống.
- **Khả năng mở rộng và tối ưu hiệu năng:** PostgreSQL cho phép triển khai các cơ chế chỉ mục và tối ưu truy vấn nâng cao, phù hợp với bài toán tìm kiếm và gợi ý trên tập dữ liệu lớn.

Nhờ cách tiếp cận này, kho dữ liệu không chỉ đóng vai trò lưu trữ, mà còn trở thành nền tảng xử lý trung tâm cho các tác vụ suy luận ngữ nghĩa của hệ thống.

5.2 Thiết kế lược đồ dữ liệu quan hệ



Hình 5.1: Lược đồ thực thể-quan hệ của kho dữ liệu

Lược đồ cơ sở dữ liệu được thiết kế dựa trên mô hình **thực thể – quan hệ (Entity–Relationship Model)**, nhằm đảm bảo tính nhất quán, khả năng mở rộng và hiệu quả truy vấn. Các thực thể chính trong kho dữ liệu bao gồm:

- **Papers (Bài báo):** Lưu trữ thông tin cốt lõi của các công trình nghiên cứu khoa học.
- **Venues (Nơi xuất bản):** Đại diện cho các tạp chí khoa học và hội nghị. Venues là đối tượng đích trong bài toán đề xuất nơi nộp bài.
- **Authors (Tác giả):** Lưu trữ thông tin về các nhà nghiên cứu.
- **Keywords:** Biểu diễn các chủ đề hoặc từ khóa nghiên cứu liên quan đến bài báo.

Các mối quan hệ giữa các thực thể được mô hình hóa chặt chẽ nhằm phản ánh đúng bản chất của dữ liệu học thuật:

- Quan hệ **Many-to-One** giữa *Paper* và *Venue*.
- Quan hệ **Many-to-Many** giữa *Paper* và *Author*, thông qua bảng trung gian.
- Quan hệ **Many-to-Many** giữa *Paper* và *Keyword*.

Thiết kế này cho phép hệ thống vừa truy vấn chi tiết từng bài báo, vừa tổng hợp và phân tích dữ liệu theo tác giả, chủ đề hoặc nơi xuất bản.

5.3 Lưu trữ và quản lý dữ liệu vector ngữ nghĩa

Một điểm trọng tâm trong thiết kế kho dữ liệu của dự án là **tích hợp khả năng lưu trữ và truy vấn vector ngữ nghĩa trực tiếp trong PostgreSQL** thông qua phần mở rộng *pgvector*.

Sau giai đoạn tiền xử lý và chuẩn hóa, các trường văn bản có giá trị ngữ nghĩa cao, cụ thể là *Title* và *Abstract* của bài báo, được đưa qua mô hình ngôn ngữ dựa trên kiến trúc Transformer để sinh ra các vector embedding chiều cao. Các vector này được lưu trữ trực tiếp trong bảng *Papers* dưới dạng thuộc tính kiểu *vector*.

Việc lưu trữ embedding ngay trong kho dữ liệu quan hệ mang lại các lợi ích sau:

- **Đồng nhất dữ liệu:** Dữ liệu nghiệp vụ và dữ liệu ngữ nghĩa được quản lý trong cùng một hệ thống, giảm độ phức tạp trong vận hành.
- **Hỗ trợ tìm kiếm ngữ nghĩa hiệu quả:** Các phép đo khoảng cách như *Cosine Similarity* hoặc *Euclidean Distance* có thể được thực hiện trực tiếp trong truy vấn SQL.
- **Khả năng kết hợp truy vấn:** Hệ thống cho phép kết hợp điều kiện lọc theo metadata (ví dụ: lĩnh vực, năm xuất bản, loại tạp chí) với truy vấn tương đồng ngữ nghĩa trong cùng một truy vấn, phục vụ hiệu quả cho bài toán gợi ý.

Cách tiếp cận này đặc biệt phù hợp với các hệ thống gợi ý học thuật có quy mô vừa và lớn, nơi yêu cầu cân bằng giữa hiệu năng, tính đơn giản và khả năng mở rộng.

5.4 Chiến lược tối ưu hóa hiệu năng kho dữ liệu

Trước khối lượng lớn dữ liệu học thuật và yêu cầu truy vấn ngữ nghĩa có độ trễ thấp, hệ thống áp dụng nhiều chiến lược tối ưu hóa hiệu năng ở tầng kho dữ liệu.

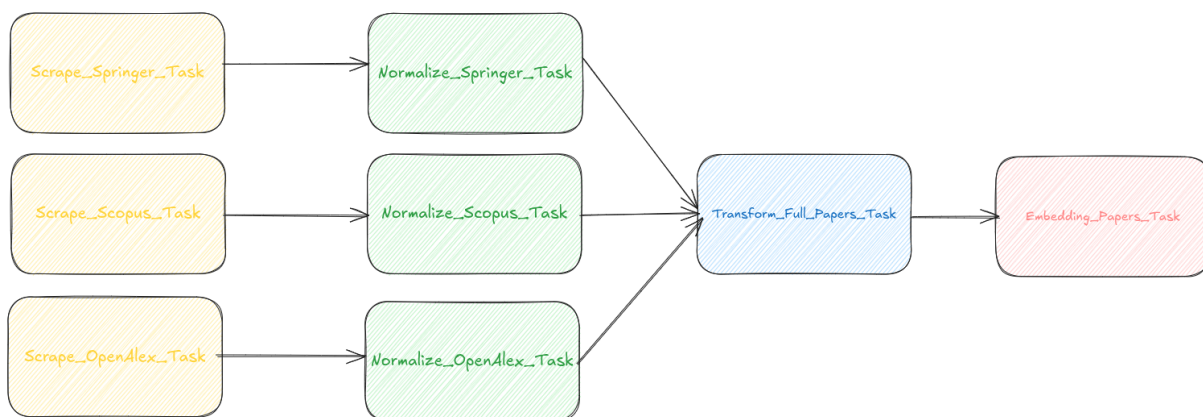
- **Tối ưu truy vấn metadata:** Các chỉ mục B-Tree được xây dựng trên các trường thường xuyên được sử dụng trong điều kiện lọc như DOI, tên nơi xuất bản và năm xuất bản.
- **Tối ưu tìm kiếm vector:** Đối với các cột embedding, hệ thống sử dụng các cấu trúc chỉ mục tìm kiếm gần đúng (*Approximate Nearest Neighbor – ANN*) như **HNSW** hoặc **IVFFlat**, giúp giảm đáng kể thời gian truy vấn khi so sánh vector trên tập dữ liệu lớn.
- **Định hướng mở rộng trong tương lai:** Kiến trúc kho dữ liệu được thiết kế sẵn sàng cho các kỹ thuật phân mảnh dữ liệu (*partitioning*) theo thời gian hoặc lĩnh vực nghiên cứu, nhằm duy trì hiệu năng khi quy mô dữ liệu tiếp tục tăng trưởng.

Chương 6

Điều phối và tự động hóa xử lý (Pipelines & Orchestration)

6.1 Mô hình điều phối dựa trên đồ thị DAG

Trong các hệ thống xử lý dữ liệu học thuật quy mô lớn, việc đảm bảo thứ tự thực thi và tính nhất quán của dữ liệu đóng vai trò then chốt. Do đó, hệ thống áp dụng mô hình điều phối dựa trên **Đồ thị không chu trình có hướng (Directed Acyclic Graph – DAG)** để biểu diễn toàn bộ luồng xử lý dữ liệu.



Hình 6.1: Lược đồ thực thể–quan hệ của kho dữ liệu

Hình trên minh họa DAG của hệ thống, trong đó mỗi đỉnh đại diện cho một tác vụ xử lý độc lập, còn mỗi cạnh biểu diễn quan hệ phụ thuộc dữ liệu giữa các tác vụ. Cấu trúc không chu trình của DAG đảm bảo rằng mỗi chu kỳ pipeline chỉ tiến về phía trước, loại bỏ khả năng xử lý lặp hoặc trạng thái dữ liệu không xác định.

6.2 Các giai đoạn xử lý

Dựa trên cấu trúc DAG, pipeline được tổ chức thành ba giai đoạn xử lý chính, tương ứng với các cụm node trong đồ thị:

- **Giai đoạn thu thập dữ liệu (Ingestion):** Bao gồm các tác vụ thu thập dữ liệu từ các nguồn học thuật khác nhau, tạo ra tập dữ liệu đầu vào cho toàn bộ pipeline.

- **Giai đoạn chuẩn hóa (Normalization):** Thực hiện làm sạch, hợp nhất và chuẩn hóa dữ liệu nhằm đưa các nguồn dị biệt về một mô hình dữ liệu thống nhất.
- **Giai đoạn biến đổi (Transformation):** Thực hiện biến đổi dữ liệu dạng JSON thành các thực thể riêng biệt, lưu trữ thành các bảng cho mục đích truy vấn và phân tích.
- **Giai đoạn vector hóa (Embedding):** Chuyển đổi nội dung văn bản học thuật thành biểu diễn vector liên tục, phục vụ cho các tác vụ tìm kiếm và gợi ý ngữ nghĩa.

Việc phân rã pipeline theo các giai đoạn rõ ràng giúp DAG phản ánh trực tiếp kiến trúc xử lý của hệ thống.

6.3 Quản lý phụ thuộc và đảm bảo tính nhất quán dữ liệu

Các quan hệ phụ thuộc được thể hiện trong DAG đảm bảo rằng mỗi tác vụ chỉ được kích hoạt khi toàn bộ dữ liệu đầu vào đã sẵn sàng. Điều này cho phép hệ thống duy trì **tính nhất quán theo lô (batch consistency)**, trong đó mỗi lần thực thi pipeline tạo ra một trạng thái dữ liệu đầy đủ và khép kín từ thu thập đến vector hóa.

Bên cạnh đó, mô hình DAG tạo điều kiện cho việc mở rộng pipeline trong tương lai, thông qua việc bổ sung các node xử lý mới mà không làm phá vỡ cấu trúc điều phối hiện có.

Chương 7

Mô hình đề xuất hội nghị và tạp chí khoa học (Conference & Journal Recommendation Model)

Trong phần này, chúng tôi trình bày quá trình xây dựng và đánh giá mô hình đề xuất hội nghị và tạp chí khoa học cho các bài nghiên cứu. Mục tiêu của mô hình là xác định các venue phù hợp nhất dựa trên nội dung học thuật của bài báo, từ đó hỗ trợ tác giả trong việc lựa chọn nơi công bố phù hợp.

Để đảm bảo tính khách quan và lựa chọn được phương pháp tối ưu cho triển khai thực tế, chúng tôi tiến hành xây dựng và so sánh **hai hướng tiếp cận khác nhau** cho bài toán đề xuất. Việc so sánh này cho phép đánh giá hiệu năng của từng mô hình dựa trên các tiêu chí đo lường thống nhất, từ đó lựa chọn mô hình có chất lượng đề xuất tốt nhất để đưa vào môi trường production.

Cụ thể, hai phương pháp được triển khai trong nghiên cứu này bao gồm:

- **Content-Based Filtering using Weighted Vector Space Model** – một phương pháp truyền thống, dựa trên độ tương đồng ngữ nghĩa giữa bài nghiên cứu và các hội nghị/tạp chí thông qua biểu diễn vector có trọng số.
- **Hybrid Search Strategy & Reciprocal Rank Fusion (RRF)** – một phương pháp hiện đại hơn, kết hợp nhiều chiến lược tìm kiếm và hợp nhất thứ hạng nhằm tận dụng ưu điểm của từng nguồn thông tin, qua đó cải thiện chất lượng kết quả đề xuất.

Thông qua việc triển khai và đánh giá hai phương pháp trên cùng một tập dữ liệu, nghiên cứu này nhằm làm rõ ưu nhược điểm của từng mô hình, đồng thời xác định chiến lược phù hợp nhất cho hệ thống đề xuất hội nghị/tạp chí khoa học trong bối cảnh ứng dụng thực tế.

7.1 Data Preprocessing

Trong hệ thống Recommendation System, bước tiền xử lý dữ liệu đóng vai trò rất quan trọng nhằm đảm bảo các vector embedding được tạo ra phản ánh đúng nội dung ngữ nghĩa của từng bài nghiên cứu.

Trong phạm vi đề tài này, chỉ ba trường dữ liệu chính được sử dụng để vector hóa, bao gồm: **title**, **abstract** và **keywords**. Mỗi trường có đặc điểm riêng nên cần áp dụng chiến lược xử lý khác nhau.

7.1.1 Preprocessing cho trường Title và Abstract

Hai trường *title* và *abstract* được xử lý nhằm loại bỏ các thành phần không mang giá trị ngữ nghĩa và chuẩn hóa văn bản trước khi đưa vào quá trình vector hóa. Việc tiền xử lý giúp giảm nhiễu và đảm bảo tính nhất quán trong biểu diễn nội dung của bài nghiên cứu.

Các bước xử lý chính bao gồm:

- Chuẩn hóa văn bản về chữ thường (lowercase)
- Loại bỏ công thức toán học, biểu thức LaTeX và nội dung trong dấu ngoặc
- Xóa URL và các ký tự đặc biệt không cần thiết
- Chuẩn hóa khoảng trắng trong văn bản

7.1.2 Preprocessing cho trường Keywords

Trường *keywords* được biểu diễn dưới dạng danh sách các từ khóa, trong đó có thể tồn tại trùng lặp ngữ nghĩa hoặc đồng thời xuất hiện cả dạng đầy đủ và dạng viết tắt của cùng một khái niệm. Do đó, quá trình tiền xử lý được thiết kế nhằm làm sạch và tinh gọn danh sách keyword trước khi vector hóa.

Các bước xử lý chính bao gồm:

- Loại bỏ nội dung dư thừa trong dấu ngoặc và các keyword rỗng
- Xác định và ưu tiên giữ lại các keyword dạng đầy đủ (multi-word phrases)
- Phát hiện và loại bỏ các từ viết tắt trùng ngữ nghĩa với dạng đầy đủ
- Chuẩn hóa toàn bộ keyword về chữ thường

7.2 Content-Based Filtering using Weighted Vector Space Model

Để xây dựng hệ thống gợi ý dựa trên nội dung, chúng tôi tập trung khai thác thông tin ngữ nghĩa từ ba trường dữ liệu đặc trưng nhất của bài báo: Title (Tiêu đề), Abstract (Tóm tắt), và Keywords (Từ khóa). Quy trình xử lý được chia thành các giai đoạn chính sau:

7.2.1 Feature Selection with Chi-Square Statistic

Trong khi *Title* và *Keywords* chứa các từ khóa cô đọng và mang tính đại diện cao, trường *Abstract* thường chứa lượng văn bản lớn với nhiều từ phổ thông hoặc nhiễu (noise), ít có giá trị phân loại. Để tối ưu hóa không gian vector và giảm chiều dữ liệu (dimensionality reduction), chúng tôi áp dụng kiểm định **Chi-square** (χ^2) để lọc và giữ lại các đặc trưng (terms) có khả năng phân loại tốt nhất đối với từng Venue (Conference/Journal).

$$\chi^2 = \frac{N(AD - BC)^2}{(A + B)(C + D)(A + C)(B + D)} \quad (7.1)$$

Trong đó:

- N là tổng số lượng bài báo ($N = A + B + C + D$).
- A là số bài báo trong journal j / conference c có chứa token.
- B là số bài báo không thuộc journal j / conference c có chứa token.
- C là số bài báo trong journal j / conference c không chứa token.
- D là số bài báo không thuộc journal j / conference c không chứa token.

Cơ chế chọn lọc:

Nếu χ^2 cao $\rightarrow$ token có liên hệ mạnh với venue đó $\rightarrow$ nên giữ lại.

Nếu χ^2 thấp $\rightarrow$ token không mang tính phân biệt $\rightarrow$ nên loại bỏ.

7.2.2 TF-IDF Vectorization & Multi-field Weighting

Sau bước chọn lọc đặc trưng, dữ liệu văn bản được chuyển đổi sang dạng vector số học sử dụng phương pháp **TF-IDF (Term Frequency - Inverse Document Frequency)**.

$$M_{title}, M_{abstract}, M_{keyword}$$

Ma trận biểu diễn cuối cùng V_{doc} :

$$V_{doc} = [w_t \cdot M_{title} \parallel w_k \cdot M_{keyword} \parallel w_a \cdot M_{abstract}]$$

7.2.3 Online Inference & Recommendation

Độ tương đồng được tính bằng Cosine Similarity:

$$\text{Similarity}(V_{query}, V_{doc}) = \frac{V_{query} \cdot V_{doc}}{\|V_{query}\| \|V_{doc}\|}$$

7.3 Hybrid Search Strategy & Reciprocal Rank Fusion (RRF)

Để tối ưu hóa độ chính xác của hệ thống gợi ý, chúng tôi áp dụng chiến lược **Hybrid Search** (Tìm kiếm lai), kết hợp sức mạnh của **Semantic Search** (Tìm kiếm ngữ nghĩa dựa trên Vector) và **Lexical Search** (Tìm kiếm từ khóa dựa trên Full-text search). Kết quả từ hai luồng tìm kiếm này được tổng hợp và xếp hạng lại bằng thuật toán **Reciprocal Rank Fusion (RRF)**.

7.3.1 Data Embedding Generation (Offline Pipeline)

Trước khi đưa vào vận hành, toàn bộ dữ liệu bài báo (Papers) trong cơ sở dữ liệu cần được chuyển đổi sang dạng vector. Quá trình này bao gồm các bước:

1. Contextual Data Preparation: Thay vì chỉ nối chuỗi các trường thông tin một cách đơn giản, chúng tôi áp dụng kỹ thuật **Structured Prompting** để tối ưu hóa khả năng hiểu ngữ cảnh của model embedding. Các trường **Title**, **Abstract**, và **Keyword** sau khi qua bước tiền xử lý (preprocessing) sẽ được kết hợp theo định dạng:

Title: {title} [SEP] Abstract: {abstract} [SEP] Keyword: {keywords}

Giải thích kỹ thuật: Việc sử dụng các tiền tố định danh (như “Title:”, “Abstract:”) kết hợp với token phân tách đặc biệt [SEP] giúp model phân biệt rõ ràng ranh giới và vai trò ngữ nghĩa của từng phần thông tin. Điều này đặc biệt quan trọng đối với các model embedding hiện đại (như Google Gemini), giúp chúng hiểu cấu trúc văn bản tốt hơn, tránh việc ngữ nghĩa của tiêu đề bị “trôi” hoặc lẫn lộn vào phần tóm tắt, từ đó tạo ra vector đại diện chính xác hơn cho bài báo.

2. Batch Processing & Task Specification: Do khối lượng dữ liệu lớn và giới hạn về Rate Limit của API, quy trình tạo embedding được thực hiện theo cơ chế **Batch Processing** (kích thước batch = 100). Đặc biệt, khi gọi API của Google (`text-embedding-004`), chúng tôi cấu hình tham số `task_type="RETRIEVAL_DOCUMENT"`.

Mục đích: Tham số này thông báo cho model rằng vector đang được tạo đóng vai trò là “tài liệu” trong kho dữ liệu, giúp model tối ưu hóa vector cho tác vụ truy xuất thông tin không đối xứng (Asymmetric Retrieval).

```
contents = [{"parts": [{"text": t}]} for t in texts]
response = self._call_embed_api(contents, "RETRIEVAL_DOCUMENT")
return [emb.values for emb in response.embeddings]
```

7.3.2 User Input Processing (Online Inference)

Khi người dùng nhập thông tin đầu vào (Title, Abstract, Keywords), hệ thống sẽ xử lý song song qua hai luồng riêng biệt để phục vụ cho Hybrid Search:

Luồng 1: Vector Search Preparation (Semantic) Dữ liệu đầu vào được xử lý và gộp theo format tương tự như trong Database:

Title: {title} [SEP] Abstract: {abstract} [SEP] Keyword: {keywords}

Đồng thời, khi gọi API embedding cho user input, chúng tôi sử dụng `task_type="RETRIEVAL_QUERY"`. Việc phân biệt task type giữa “Document” (Database) và “Query” (User Input) giúp model điều chỉnh không gian vector để tối đa hóa độ tương đồng giữa câu hỏi và tài liệu trả lời.

```
contents = [{"parts": [{"text": text}]}]
response = self._call_embed_api(contents, "RETRIEVAL_QUERY")
return np.array(response.embeddings[0].values)
```

Luồng 2: Keyword Search Preparation (Lexical) Đối với luồng tìm kiếm từ khóa (Full-text search), dữ liệu đầu vào được gộp đơn giản bằng cách nối chuỗi:

```
{title} {abstract} {keywords}
```

Giải thích kỹ thuật: Khác với Vector Search (hiểu ngữ nghĩa sâu), Keyword Search (dựa trên Postgres `tsvector`) hoạt động dựa trên việc khớp từ vựng (lexical matching). Việc thêm các thẻ meta như “Title:” hay “[SEP]” vào luồng này là không cần thiết và có thể gây nhiễu (noise), khiến thuật toán BM25 đánh giá sai trọng số hoặc không tìm thấy từ khóa chính xác. Do đó, chúng tôi giữ input ở dạng văn bản thuần túy (clean text) để tối ưu hóa hiệu quả của chỉ mục đảo ngược (Inverted Index).

7.3.3 Retrieval & Ranking Mechanism

Quy trình tìm kiếm và xếp hạng diễn ra như sau:

1. **Vector Search Execution:** Vector truy vấn của người dùng được so sánh với vector của toàn bộ bài báo trong Database sử dụng độ đo **Cosine Similarity** thông qua chỉ mục HNSW. Hệ thống trích xuất Top-100 ứng viên có ngữ nghĩa gần nhất.
2. **Keyword Search Execution:** Chuỗi văn bản truy vấn được so sánh với cột `tsv` (đã được đánh chỉ mục GIN trong Postgres) sử dụng thuật toán xếp hạng **ts_rank** (dựa trên **BM25**). Hệ thống trích xuất Top-100 ứng viên có độ khớp từ khóa cao nhất.
3. **Reciprocal Rank Fusion (RRF):** Hai danh sách kết quả trên được gộp lại. Điểm số RRF được tính toán dựa trên thứ hạng (rank) của bài báo trong mỗi danh sách con, giúp cân bằng giữa những bài báo khớp chính xác từ khóa và những bài báo có nội dung liên quan về mặt ý nghĩa.
4. **Final Recommendation:** Danh sách kết quả cuối cùng được sắp xếp theo điểm RRF giảm dần. Hệ thống sẽ trả về Top-10 Venue (Conference/Journal) phù hợp nhất dựa trên các bài báo này để gợi ý cho người dùng.

Chương 8

Phát triển ứng dụng web (Web Application)

8.1 Kiến trúc tổng thể ứng dụng

Ứng dụng được xây dựng theo mô hình **Client–Server**, trong đó Frontend và Backend được tách biệt hoàn toàn và giao tiếp thông qua **RESTful API**. Cách tổ chức này đảm bảo tính rõ ràng trong phân tầng, đồng thời tạo điều kiện cho việc mở rộng hệ thống trong các giai đoạn sau.

Frontend chịu trách nhiệm thu thập thông tin bản thảo và hiển thị kết quả gợi ý, trong khi Backend đảm nhiệm toàn bộ logic xử lý nghiệp vụ, bao gồm phân tích nội dung, truy vấn dữ liệu và suy luận gợi ý. Việc phân tách này giúp hệ thống duy trì tính linh hoạt và giảm sự phụ thuộc chặt chẽ giữa giao diện và thuật toán.

8.2 Tầng Backend và tổ chức xử lý nghiệp vụ

Tầng Backend được phát triển bằng **FastAPI**, phù hợp với đặc thù của hệ thống xử lý dữ liệu và học máy. Các API được thiết kế theo hướng đơn giản, tập trung vào việc tiếp nhận dữ liệu đầu vào (bản thảo người dùng) và trả về danh sách gợi ý tương ứng.

Logic xử lý được tổ chức theo mô hình dịch vụ, trong đó các bước như tiền xử lý văn bản, gọi mô hình embedding và truy vấn vector được tách khỏi lớp định nghĩa API. Cách tiếp cận này giúp luồng xử lý rõ ràng, dễ theo dõi và thuận tiện cho việc mở rộng hoặc thay thế từng thành phần xử lý trong tương lai.

8.3 Tầng Frontend và giao diện người dùng

Frontend được xây dựng bằng **Next.js** kết hợp với **TypeScript**, hướng tới việc đảm bảo tính nhất quán dữ liệu và trải nghiệm người dùng ổn định. Các thành phần giao diện được thiết kế theo hướng tái sử dụng, phản ánh trực tiếp các chức năng chính của hệ thống: nhập nội dung bản thảo và hiển thị kết quả gợi ý.

Việc sử dụng cơ chế render phía server cho phép giao diện được hiển thị sớm, trong khi các tác vụ xử lý phía sau vẫn đang được thực hiện. Điều này giúp cải thiện cảm nhận phản hồi ban đầu, đặc biệt trong các tác vụ có độ trễ tính toán cao.

8.4 Quản lý độ trễ và khả năng diễn giải kết quả

Do hệ thống dựa trên tìm kiếm ngữ nghĩa và vector hóa văn bản, độ trễ xử lý là yếu tố không thể tránh khỏi. Thay vì tối ưu hóa thuần túy về thời gian tính toán, ứng dụng tập trung vào việc **giảm độ trễ cảm nhận** thông qua phản hồi giao diện theo từng trạng thái xử lý.

Kết quả gợi ý được trình bày kèm theo các chỉ báo ngữ cảnh như mức độ tương đồng hoặc thông tin liên quan đến nội dung, giúp người dùng hiểu được cơ sở của các đề xuất. Điều này góp phần nâng cao tính minh bạch và độ tin cậy của hệ thống trong bối cảnh sử dụng học thuật.

Chương 9

Triển khai và vận hành hệ thống (DevOps)

9.1 Chiến lược đóng gói và đảm bảo tính nhất quán môi trường

Một thách thức phổ biến trong quá trình phát triển các hệ thống phần mềm đa thành phần là sự khác biệt giữa môi trường phát triển và môi trường triển khai thực tế. Để giảm thiểu rủi ro này và đảm bảo khả năng tái lập hệ thống, toàn bộ dự án được đóng gói và triển khai thống nhất bằng công nghệ Docker.

Mỗi thành phần chính của hệ thống được xây dựng với Dockerfile riêng, phản ánh đúng đặc thù của từng phân hệ. Cách tiếp cận này cho phép cô lập môi trường chạy, kiểm soát chính xác phiên bản thư viện, đồng thời đơn giản hóa quá trình triển khai trên các máy chủ khác nhau.

Việc quản lý phụ thuộc được thực hiện thông qua các tệp cấu hình chuẩn như `requirements.txt` (Backend) và `package.json` (Frontend), giúp đảm bảo rằng hệ thống có thể được build lại một cách nhất quán tại bất kỳ thời điểm nào.

9.2 Điều phối dịch vụ với Docker Compose

Do hệ thống bao gồm nhiều dịch vụ chạy song song (Backend API, Frontend và Database), Docker Compose được sử dụng như công cụ điều phối trung tâm. Cách tổ chức này cho phép khởi động toàn bộ hệ thống chỉ với một lệnh duy nhất, đồng thời đảm bảo các dịch vụ có thể giao tiếp với nhau thông qua mạng nội bộ.

Các tệp cấu hình Docker Compose được phân tách theo mục đích sử dụng:

- Tệp cấu hình chính mô tả các dịch vụ cốt lõi và mối quan hệ phụ thuộc giữa chúng.
- Tệp cấu hình cho môi trường phát triển hỗ trợ ánh xạ mã nguồn từ máy host vào container, giúp rút ngắn vòng lặp chỉnh sửa và kiểm thử.
- Tệp cấu hình cho môi trường triển khai tập trung vào việc chạy ổn định các container đã được build sẵn, không bật các chế độ phục vụ cho debug.

Cách tiếp cận này giúp cân bằng giữa tính linh hoạt trong phát triển và sự ổn định khi vận hành.

9.3 Tự động hóa quy trình triển khai

Để giảm thiểu thao tác thủ công và hạn chế lỗi phát sinh trong quá trình triển khai, dự án sử dụng các tập lệnh shell đơn giản nhằm tự động hóa các bước khởi tạo hệ thống. Các tập lệnh này hỗ trợ:

- Kiểm tra sự tồn tại của các biến môi trường cần thiết.
- Khởi động các container theo đúng thứ tự phụ thuộc.
- Khởi tạo cấu trúc dữ liệu ban đầu trong cơ sở dữ liệu thông qua các script SQL.

Nhờ đó, quá trình triển khai trở nên nhất quán và có thể lặp lại, ngay cả khi thực hiện trên các máy chủ hoặc môi trường khác nhau.

9.4 Quản lý dữ liệu và bảo mật cấu hình

Dữ liệu của hệ thống được lưu trữ bền vững thông qua cơ chế volume của Docker, cho phép ánh xạ dữ liệu từ container ra ổ đĩa của máy host. Điều này đảm bảo dữ liệu không bị mất khi container bị dừng hoặc khởi động lại.

Các thông tin nhạy cảm như khóa API hoặc mật khẩu cơ sở dữ liệu không được ghi cứng trong mã nguồn mà được quản lý thông qua biến môi trường và các tập `.env`. Những tập này được loại trừ khỏi hệ thống quản lý phiên bản, tuân thủ các nguyên tắc bảo mật cơ bản trong phát triển phần mềm.

Ngoài ra, các dịch vụ trong hệ thống đều ghi log ra chuẩn `stdout/stderr`, cho phép người vận hành dễ dàng theo dõi trạng thái hoạt động và phát hiện lỗi trong quá trình chạy.

9.5 Đánh giá khả năng mở rộng ở mức thiết kế

Trong phạm vi của đề án, hệ thống được triển khai ở quy mô đơn giản, phù hợp với mục tiêu nghiên cứu và thử nghiệm. Tuy nhiên, việc tổ chức hệ thống theo các dịch vụ độc lập và đóng gói bằng container đã tạo tiền đề cho khả năng mở rộng trong tương lai.

Cách tiếp cận này cho phép từng dịch vụ có thể được triển khai hoặc nâng cấp độc lập mà không ảnh hưởng đến toàn bộ hệ thống, đồng thời giữ cho cấu trúc logic của ứng dụng rõ ràng và dễ bảo trì.

Chương 10

Hạn chế và hướng phát triển

10.1 Hạn chế của hệ thống

Mặc dù hệ thống đã đáp ứng được các mục tiêu nghiên cứu ban đầu, một số hạn chế vẫn tồn tại do ràng buộc về dữ liệu, tài nguyên và phạm vi triển khai.

Thứ nhất, **hạn chế về dữ liệu** là yếu tố ảnh hưởng trực tiếp đến chất lượng gợi ý của hệ thống. Tập dữ liệu hiện tại chưa đủ dồi dào cả về số lượng lẫn mức độ đa dạng, đặc biệt đối với các lĩnh vực chuyên sâu hoặc mang tính liên ngành. Điều này khiến mô hình khó học được đầy đủ đặc trưng ngữ nghĩa và có thể làm giảm độ chính xác của kết quả gợi ý trong một số trường hợp.

Thứ hai, **hạn chế về tài nguyên** (tính toán, lưu trữ và thời gian phát triển) đã giới hạn khả năng triển khai các phương án xử lý nâng cao. Nhiều kỹ thuật và mô hình đã được nghiên cứu ở mức lý thuyết nhưng chưa thể áp dụng đầy đủ trong hệ thống thực nghiệm do yêu cầu cao về hạ tầng và chi phí vận hành.

Thứ ba, **hạn chế về thuật toán** thể hiện ở việc hệ thống hiện mới khai thác một phần các phương pháp đã được khảo sát. Các hướng tiếp cận phức tạp hơn, có khả năng cải thiện đáng kể chất lượng gợi ý, chưa được tích hợp do phụ thuộc chặt chẽ vào dữ liệu quy mô lớn và tài nguyên tính toán mạnh hơn.

Những hạn chế này phản ánh đúng bối cảnh của một đồ án nghiên cứu – nơi việc cân bằng giữa mục tiêu học thuật và khả năng triển khai thực tế là yếu tố then chốt.

10.2 Hướng phát triển trong tương lai

Dựa trên các hạn chế đã được xác định, hệ thống có thể được phát triển theo những hướng sau.

Trước hết, việc **mở rộng và làm giàu dữ liệu** là ưu tiên hàng đầu. Bổ sung thêm dữ liệu từ nhiều nguồn và nhiều lĩnh vực khác nhau sẽ giúp cải thiện độ bao phủ, tăng khả năng khái quát và nâng cao chất lượng gợi ý của hệ thống.

Tiếp theo, hệ thống có thể được **tăng cường về mặt thuật toán** bằng cách áp dụng đầy đủ hơn các kiến thức đã được nghiên cứu, thử nghiệm các mô hình nâng cao và tối ưu quy trình xử lý. Điều này cho phép đánh giá sâu hơn hiệu quả của các phương pháp trong điều kiện gần với thực tế sử dụng.

Cuối cùng, một hướng phát triển quan trọng là **định hướng sản phẩm**, trong đó hệ

thống được hoàn thiện thành một nguyên mẫu (prototype) hoặc sản phẩm khả thi tối thiểu (MVP). Để hiện thực hóa hướng đi này, cần có sự đầu tư đồng bộ về tài nguyên, dữ liệu theo từng lĩnh vực chuyên biệt (domain), cũng như quá trình tinh chỉnh về giao diện và trải nghiệm người dùng.

Hướng phát triển này không chỉ mở rộng phạm vi ứng dụng của hệ thống mà còn tạo điều kiện để kiểm chứng giá trị của giải pháp trong bối cảnh thực tế, vượt ra ngoài khuôn khổ nghiên cứu học thuật.

Chương 11

Kết luận và đánh giá

Báo cáo seminar tập trung vào việc thiết kế và triển khai một hệ thống hỗ trợ lựa chọn nơi nộp bài khoa học dưới dạng một ứng dụng web. Trọng tâm của seminar không nằm ở việc đề xuất thuật toán mới, mà ở việc vận dụng có chọn lọc các kiến thức đã được nghiên cứu để xây dựng một giải pháp có khả năng sử dụng trong thực tế.

Thông qua quá trình triển khai, seminar đã làm rõ cách một bài toán mang tính nghiệp vụ trong lĩnh vực xuất bản học thuật có thể được mô hình hóa, xử lý và hiện thực hóa thành một hệ thống hoàn chỉnh. Cách tiếp cận này nhấn mạnh vai trò của thiết kế hệ thống, tổ chức dữ liệu và khả năng tích hợp giữa các thành phần kỹ thuật.

11.1 Những kết quả và đóng góp đạt được

Một kết quả quan trọng của seminar là việc xây dựng được một quy trình xử lý dữ liệu và gợi ý có cấu trúc rõ ràng. Dữ liệu học thuật được thu thập, tiền xử lý và tổ chức theo pipeline thống nhất, tạo điều kiện cho hệ thống vận hành ổn định và dễ bảo trì. Cách tiếp cận này phản ánh tư duy triển khai thực tế, phù hợp với các hệ thống ứng dụng dựa trên dữ liệu.

Bên cạnh đó, seminar đã áp dụng các kỹ thuật biểu diễn ngữ nghĩa để cải thiện chất lượng gợi ý so với các phương pháp so khớp đơn giản. Việc sử dụng vector embedding cho phép hệ thống khai thác mối liên hệ ngữ nghĩa giữa nội dung bài viết và phạm vi của các venue, qua đó nâng cao tính hữu ích của kết quả đầu ra trong bối cảnh ứng dụng.

Ngoài ra, toàn bộ giải pháp đã được hiện thực hóa dưới dạng một ứng dụng web, với kiến trúc phân tách rõ ràng giữa các thành phần xử lý, giao diện và dữ liệu. Điều này cho thấy seminar không chỉ dừng lại ở mức ý tưởng, mà đã giải quyết được các vấn đề triển khai và vận hành cơ bản của một hệ thống ứng dụng.

11.2 Đánh giá hệ thống

Trong phạm vi thử nghiệm, hệ thống hoạt động ổn định và đáp ứng được các yêu cầu chức năng đã đặt ra. Các luồng xử lý chính được triển khai đầy đủ, từ tiếp nhận dữ liệu đầu vào đến trả về kết quả gợi ý cho người dùng.

Mặc dù chất lượng gợi ý vẫn chịu ảnh hưởng bởi độ dồi dào của dữ liệu và giới hạn tài nguyên, hệ thống đã thể hiện được tính khả thi của hướng tiếp cận. Những giới hạn này

phù hợp với mục tiêu của seminar và không làm giảm giá trị của kết quả đạt được, mà chủ yếu phản ánh các điều kiện triển khai trong bối cảnh thực tế.

Chương 12

Lời kết

Tổng kết lại, báo cáo seminar đã hoàn thành tốt mục tiêu xây dựng và đánh giá một hệ thống ứng dụng hỗ trợ lựa chọn venue nộp bài khoa học. Kết quả đạt được cho thấy việc vận dụng các kỹ thuật xử lý dữ liệu và gợi ý vào một bài toán thực tiễn là hoàn toàn khả thi và có giá trị sử dụng.

Seminar này có thể được xem như một bước khởi đầu cho việc phát triển các hệ thống hỗ trợ nghiên cứu khoa học theo hướng ứng dụng, đồng thời tạo nền tảng cho các cải tiến tiếp theo về dữ liệu, thuật toán và trải nghiệm người dùng trong tương lai.

Tài liệu tham khảo

- Contributors (Ed.). (n.d.). Pgvector/pgvector: Open-source vector similarity search for Postgres. GitHub. <https://github.com/pgvector/pgvector>
- Danh, V. V. (n.d.). BM25 thuật toán xếp hạng các văn bản theo độ phù hợp. November 2, 2025, https://viblo.asia/p/bm25-thuat-toan-xep-hang-cac-van-ban-theo-do-phu-hop-_document-length-5
- DeepWiki. (2025, December 5). Vector type: Pgvector/pgvector. <https://deepwiki.com/pgvector/pgvector/3.1-vector-distance-metrics>
- Evidently AI Team. (n.d.). 10 metrics to evaluate recommender and ranking systems. Evidently AI - Open-Source ML Monitoring and Observability. <https://www.evidentlyai.com/ranking-metrics/evaluating-recommender-systems#predictive-quality-metrics>
- Foxgem. (n.d.-a). Vector Database Indexing: A Comprehensive Guide. November 2, 2025, <https://dev.to/foxgem/vector-database-indexing-a-comprehensive-guide-3an1>
- GeeksforGeeks. (2025, November 17). SQL Server ROW_NUMBER function with partition by. https://www.geeksforgeeks.org/sql-server/sql-server-row_number-function
- Google. (n.d.). Text embeddings API | Generative AI on Vertex AI | google cloud documentation. Google. https://docs.cloud.google.com/vertex-ai/generative-ai/docs/model-reference/text-embeddings-api#generative-ai-get-text-embedding-python.vertex_ai_sdk
- Hoang, H. T. (n.d.). Information Retrieval: Kết hợp kết quả của nhiều mô hình ranking (RRF. November 2, 2025, <https://viblo.asia/p/information-retrieval-ket-hop-ket-qua-c>
- Huynh, S. T., Dang, N., Huynh, P. T., Nguyen, D. H., Nguyen, B. T. (2021). A fusion approach for paper submission recommendation system. Lecture Notes in Computer Science, 72–83. https://doi.org/10.1007/978-3-030-79463-7_7
- Katz, J. (n.d.). Hybrid Search With PostgreSQL and Pgvector. November 2, 2025, <https://jkatz05.com/post/postgres/hybrid-search-postgres-pgvector/>
- Le, D. H., Doan, T. T., Huynh, S. T., Nguyen, B. T. (2022). SimCPSR: Simple Contrastive Learning for paper submission recommendation system. Lecture Notes in Computer Science, 51–63. https://doi.org/10.1007/978-3-031-21743-2_5

- Microsoft. (n.d.). Hybrid search scoring (RRF) - azure ai search. Hybrid search scoring (RRF) - Azure AI Search | Microsoft Learn. <https://learn.microsoft.com/en-us/azure/search/hybrid-search-ranking>
- MikeRayMSFT. (n.d.). Row_number (transact-SQL) - SQL server. SQL Server | Microsoft Learn. <https://learn.microsoft.com/en-us/sql/t-sql/functions/row-number-transact-sql?view=sql-server-ver17>
- Nguyen, D., Huynh, S., Huynh, P., Dinh, C. V., Nguyen, B. T. (2021). S2CFT: A new approach for paper submission recommendation. Lecture Notes in Computer Science, 563–573. https://doi.org/10.1007/978-3-030-67731-2_41
- Rate limits. Rate limits - Google Gemini API. (n.d.). <https://gemini-api.apidog.io/doc-965865>